Relational vs. Graph Databases

Technical Comparison

Jacopo Masia 1842884

March 9, 2026

Outline

0

- Storage Layout
- Query Complexity Analysis
- Experimental Analysis: Recipes Dataset
- Query Language Analysis
- Conclusions

Table of Contents

- Storage Layout
- Query Complexity Analysis
- Experimental Analysis: Recipes Dataset
- Query Language Analysis
- Conclusions

Storage Layout

Row / Columnar Store

Row-oriented (OLTP) — tuple contiguous in page:

id	name	age	city
1	Alice	30	Rome
2	Bob	25	Milan

Column-oriented (OLAP) — one array per column:

```
age[]:    [30, 25, 41, ...]
city[]:   [Rome, Milan, ...]
```

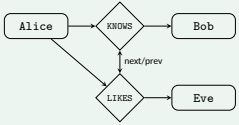
- Compression ratio ↑ on repeated values
- Analytics scan only needed columns
- Examples: Redshift, DuckDB, Parquet

Native Graph Storage (Neo4j)

Fixed-size records on disk:

- Node record: **15 B** Rel record: **34 B**

Index-free adjacency — each node holds a pointer to its *first* relationship; rels form a doubly-linked list:



- Each node holds a **direct pointer** to its first relationship record
- Properties in a **separate property store**

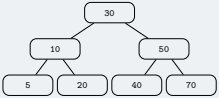
Index Structures

1 Storage Layout

Relational DB — B-Tree Index

B-Tree (default in PostgreSQL, MySQL):

- Balanced tree over a **column's values**
- Lookup, insert, delete: $O(\log n)$
- Supports equality *and* range queries



Graph DB — Index-Free Adjacency

No index needed for traversal:

- Neighbour lookup: $O(1)$ — just follow a pointer
- Cost depends on **neighbourhood size**, not graph size

Indexes still used for entry points:

- **B-Tree / Hash** — find node by property value
- **Full-text (Lucene)** — text search on properties

RDB: index resolves **every JOIN** — $O(\log n)$ per join, globally.
Graph DB: index only for the **entry point** — hops are $O(1)$ pointers.

Table of Contents

2 Query Complexity Analysis

- Storage Layout
- Query Complexity Analysis
- Experimental Analysis: Recipes Dataset
- Query Language Analysis
- Conclusions

Query Complexity: Multi-Hop Joins

Relational

k-hop query requires k self-JOINs on the edge table.

Assuming hash-join, each JOIN costs $O(n)$ where $n = |E|$.

Total: $O(k \cdot n)$ — grows with graph size, not neighbourhood size.

With index on foreign keys: $O(k \cdot d)$ per lookup (d = avg degree), but planning overhead for large k remains.

Graph

Traversal cost proportional to **neighbourhood size**, not $|V|$ or $|E|$.

k-hop traversal: $O(d^k)$ where d is average degree.

For sparse graphs ($d \ll n$): dramatically faster.

For dense super-nodes (e.g. hubs): same bottleneck. Strategies: relationship type filtering, degree caps.

Complexity Summary Table

Query Type	Relational	Graph DB
Point lookup (PK)	$O(\log n)$	$O(1)$
1-hop neighbour	$O(\log n)$ + JOIN	$O(d)$
k**-hop traversal**	$O(k \cdot n)$	$O(d^k)$
Shortest path	$O(n^2)$	$O((\lvert V\rvert + \lvert E\rvert) \log \lvert V\rvert)$
Global aggregation	$O(n)$	$O(n)$

Table of Contents

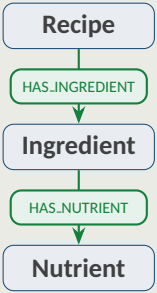
3 Experimental Analysis: Recipes Dataset

- Storage Layout
- Query Complexity Analysis
- Experimental Analysis: Recipes Dataset
- Query Language Analysis
- Conclusions

Dataset Description

3 Experimental Analysis: Recipes Dataset

Graph Schema



Dataset Sizes Tested

Scale	Recipes	Ingredients	Nutrients	R-I Edges	I-N Edges
25%	∼42.5k	4805	142	∼373k	∼304k
50%	∼85k	4869	142	∼746k	∼308k
75%	∼127.5k	4870	142	∼1.12M	∼308k
100%	∼170k	4870	142	∼1.5M	∼308k

Queries Tested

10 queries across 4 categories

Multi-hop (2-hop traversals):

- Nutrients for a recipe
- Recipes sharing an ingredient
- Filter mid-traversal by nutrient amount

Aggregation:

- Total nutrient in recipe
- Top ingredients by nutrient
- Recipes by prep time

Mixed:

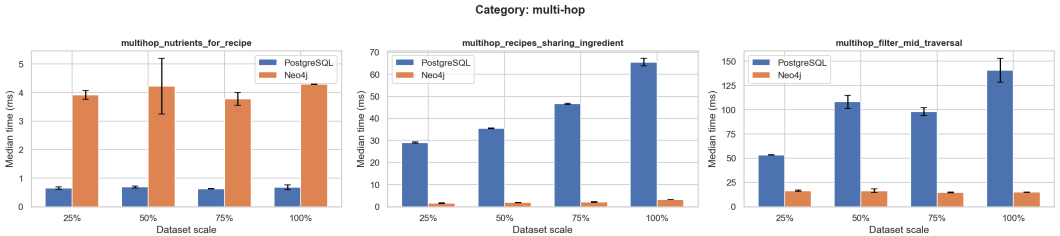
- Most used ingredients
- Subset matching given recipe
- Nutritional density

Variable-length:

- Reachable recipes in N-hops

Results: multihop queries results

3 Experimental Analysis: Recipes Dataset

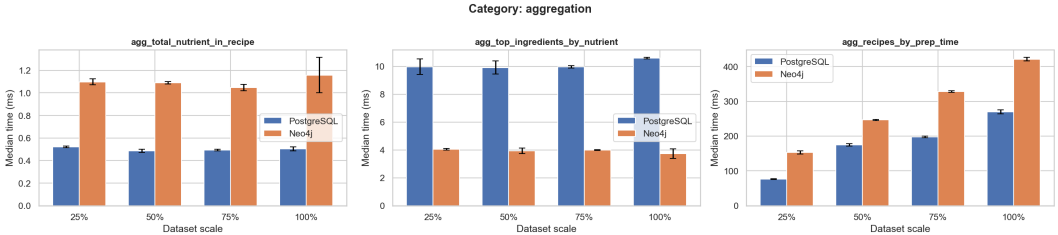


Results: multihop queries analysis

- **Nutrients for a recipe**: shallow traversal, pattern `Recipe → Ingredient → Nutrient` with specific starting recipe. Very fast queries in both cases, we are measuring the fixed per-query overhead of the two DBs, Postgres is faster.
- **Recipes sharing an ingredient**: we notice the dramatic win of Neo4j as expected, the pattern `Recipe → Ingredient ← Recipe` generates very large intermediate results (fan-out) that grows with the dataset size, meanwhile in Neo4j simply follows the pointers.
- **Filter mid-traversal by nutrient amount**: the query is to find recipes containing an ingredient with more than X amount of a given nutrient. Pattern `Recipe → Ingredient → Nutrient` with specific nutrient and amount requires SQL to make a join of the two relations across the full dataset, in Neo4j it's possible to apply the filter on the amount during the traversal.

Results: aggregation queries results

3 Experimental Analysis: Recipes Dataset

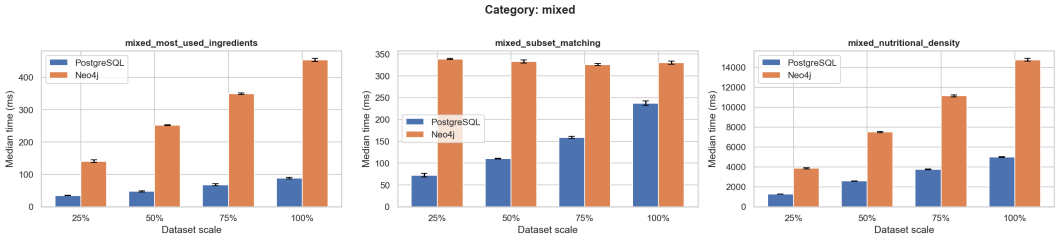


Results: aggregation queries analysis

- **Total nutrients in recipe**: simple sum with selective filter on one recipe, again we are measuring the overhead.
- **Top ingredients by nutrient**: global aggregation that requires SQL to make a full scan of the `has_nutrient` relation filtered by `nutrient_id`, even with an index on `ingredient_id` produces a very large intermediate result, meanwhile in Neo4j a traversal starts from the `Nutrient` node and reaches all the connected Ingredient nodes, a sequential scan of the local neighborhood which is cache-friendly.
- **Recipes by prep time**: range filter on a property followed by a count. Postgres uses a B-tree index on `recipe.minutes` which is ideal for range queries. Neo4j instead is not optimized for property-based range filtering which require a full node scan of all the `Recipe` nodes.

Results: mixed queries results

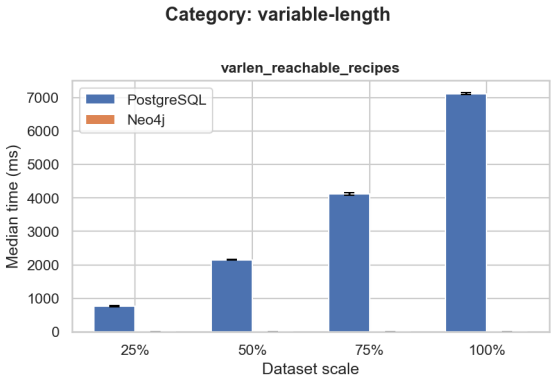
3 Experimental Analysis: Recipes Dataset



Results: mixed queries analysis

- **Most used ingredients**: classic hash aggregation on the `has_ingredient` junction table, Postgres is extremely optimized for this, Neo4j instead must traverse every `Recipe → Ingredient` relationship in the graph without pruning.
- **Subset matching given recipe**: find recipes that contain all ingredients of a given recipe. Double `NOT EXISTS` query for postgres which is optimized for that using anti-joins. Neo4j must check every required ingredient (N relationship existence) for each candidate recipe. We notice though that the computation time is constant for all dataset scales, which suggests that the cost may be dominated by a fixed overhead function (such as `collect()` or `ALL()`).
- **Nutritional density**: a 3-hop aggregation `Recipe→Ingredient→Nutrient` over the entire dataset with `AVG` and `COUNT DISTINCT`. Both systems are slow because the aggregation is global and the dataset is big, but Postgres wins because it uses hash aggregation with group-by on recipe ($O(N)$), and Neo4j must traverse every relation in the graph and the traversal step overhead of Neo4j is greater than the Postgres tuple comparison.

Results: variable length query result

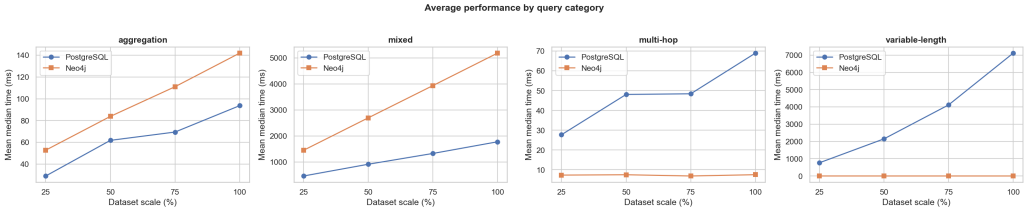


Results: variable length query analysis

- **Reachable recipes in N-hops**: here Neo4j wins by orders of magnitude (sub ms vs 7000+ ms at full scale). This is the canonical showcase for graph dbs. The query counts distinct recipes reachable within N (in this case 2 because otherwise Postgres was too slow) via shared ingredients.
  Postgres must implement BFS manually using a recursive CTE and each recursive step creates intermediate results which grow with data size.
  Neo4j has a **native variable-length path operator** (`HAS_INGREDIENT*1..N`) that maintains a compact visited set and follows relationship pointers without materializing intermediate results, pruining already visited nodes in $O(1)$.
  The reachability answer is bounded by graph topology and not global dataset size.

Results: Average Performance by Query Category

3 Experimental Analysis: Recipes Dataset



Neo4j worse | **Neo4j much worse** | **Neo4j much better** | **Neo4j much better**

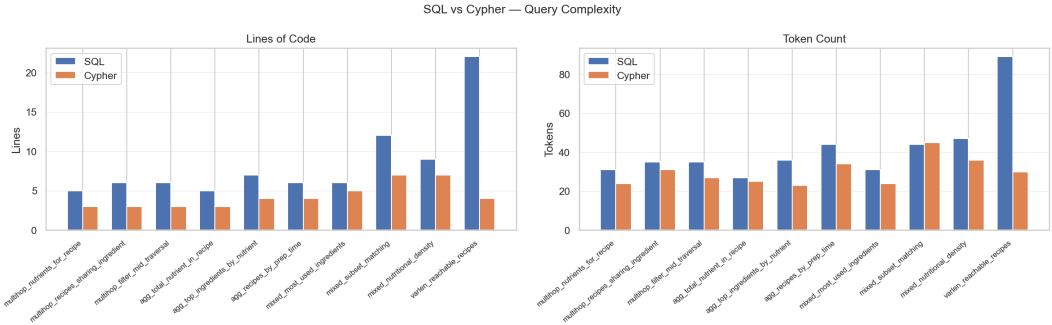
Table of Contents

4 Query Language Analysis

- Storage Layout
- Query Complexity Analysis
- Experimental Analysis: Recipes Dataset
- Query Language Analysis
- Conclusions

Query verbosity: SQL vs. Cypher

4 Query Language Analysis



Comparison

Key differences

- **Relationships**: in SQL, relationships are expressed via foreign keys and JOINs, in Cypher, they have with their own type, direction, and properties and are easier to access (`-[:REL]->`)
- **Traversal**: multi-hop queries grow linearly in SQL (one JOIN per hop), Cypher handles them natively with variable-length patterns (`[:REL*1..n]`)
- **Verbosity**: SQL tends to be more verbose as query complexity grows, Cypher stays concise because the graph structure already encodes relationships

Table of Contents

5 Conclusions

- Storage Layout
- Query Complexity Analysis
- Experimental Analysis: Recipes Dataset
- Query Language Analysis
- Conclusions

When to Choose a Relational Database

5 Conclusions

Choose a Relational DB when:

- Data is **tabular and well-structured** with a stable schema
- Workload is **transactional** (banking, e-commerce, ERP)
- Queries involve **global aggregations** or OLAP analytics
- Joins are **shallow** (1–2 levels)

Choose a Graph DB when:

- Data is **highly connected** and relationships are central
- Queries involve **multi-hop traversals** (fraud rings, social networks, supply chains)
- **Pathfinding** or **reachability** is required
- Schema evolves frequently (new relationship types)
- **Recommendation engines**
- **Knowledge graphs** or semantic data